

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Sementara
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengenalan Mikroprosesor

Bagas Aryo Dananjoyo - 5024241031

2026

1 Pendahuluan

1.1 Latar Belakang

Mikrokontroler hampir selalu ada di balik perangkat elektronik yang kita pakai sehari-hari, mulai dari remote AC, mesin cuci, sampai perangkat IoT yang terhubung ke internet. Karena itu, kemampuan memprogram mikrokontroler dan mengakses pin GPIO-nya menjadi pondasi sebelum kita bisa membuat sistem tertanam yang lebih kompleks.

Praktikum ini adalah langkah awal untuk membangun pondasi tersebut. Kita mulai dari menyiapkan lingkungan pengembangan (PlatformIO di VSCode), lalu mencoba dua board yang punya karakter berbeda: STM32 Blackpill yang diprogram lewat ST-Link, dan ESP32 yang bisa diprogram langsung lewat USB. Pada ESP32 kita juga membandingkan dua framework, yaitu Arduino yang sederhana dan ESP-IDF yang lebih native, sehingga terlihat bahwa chip yang sama bisa didekati dengan cara yang berbeda.

Selain itu, praktikum ini membahas tiga cara kerja GPIO: sebagai output digital (menyalakan LED), sebagai input digital (membaca tombol), dan sebagai input analog (membaca sensor cahaya LDR lewat ADC). Di dunia nyata, tombol yang dibaca sembarangan akan menghasilkan data yang tidak bisa dipercaya, entah karena pin yang mengambang (*floating*) atau karena getaran kontak mekanik (*bouncing*). Memahami penyebab dan cara mengatasinya sejak awal membuat program yang kita tulis lebih andal, bukan sekadar jalan di meja percobaan.

1.2 Dasar Teori

PlatformIO dan VSCode

PlatformIO adalah ekosistem pengembangan *embedded* lintas platform yang mendukung banyak board dan framework (Arduino, STM32Cube/HAL, ESP-IDF, dan lainnya) dalam satu antarmuka. Board dan framework cukup diatur lewat berkas `platformio.ini`, sehingga satu chip seperti ESP32 bisa diprogram dengan Arduino atau ESP-IDF tanpa mengganti alat. Struktur project-nya terdiri dari `platformio.ini`, folder `src/` untuk kode utama, `include/` untuk header, dan `lib/` untuk library tambahan.

STM32 Blackpill

Blackpill berbasis STM32F401CC (ARM Cortex-M4 dengan FPU, clock hingga 84 MHz, Flash 256 KB, RAM 64 KB). LED onboard-nya berada di pin PC13 dan bersifat aktif LOW, artinya LED menyala saat pin diberi LOW. Board ini tidak punya programmer bawaan, jadi diperlukan ST-Link eksternal lewat protokol SWD (SWDIO, SWCLK, GND, dan 3.3V). Dengan framework Arduino (STM32duino), fungsi seperti `pinMode()`, `digitalWrite()`, dan `delay()` bisa dipakai seperti di Arduino biasa, meskipun di baliknya tetap berdiri di atas HAL dari STMicroelectronics.

ESP32 dan ESP-IDF

ESP32 adalah mikrokontroler 32-bit dual-core (Xtensa LX6) dengan WiFi dan Bluetooth terintegrasi, serta USB-to-Serial bawaan. ESP-IDF adalah framework resmi dari Espressif yang dibangun di atas FreeRTOS. Perbedaannya dengan framework Arduino dirangkum pada Tabel 2.

Tabel 1: Perbandingan framework Arduino dan ESP-IDF

Aspek	**Framework Arduino**	**Framework ESP-IDF**
Titik masuk program	`setup()` dan `loop()`	`app_main()`
Delay/timing	`delay(ms)`	`vTaskDelay(pdMS_TO_TICKS(ms))`
Tingkat abstraksi	Tinggi, mudah dipelajari	Rendah–menengah, lebih native dan kuat
Akses fitur RTOS	Tersembunyi/terbatas	Langsung (task, semaphore, queue)

Pull-up dan Pull-down

Pin GPIO yang tidak terhubung ke level tegangan apa pun akan *floating*, sehingga nilainya acak akibat *noise*. Untuk mengatasinya dipakai resistor pull-up (pin ditarik ke 3.3V, terbaca HIGH saat tombol tidak ditekan dan LOW saat ditekan ke GND) atau pull-down (pin ditarik ke GND, logikanya kebalikan). Resistor ini bisa dipasang eksternal (umumnya 10 kΩ) atau diaktifkan lewat firmware dengan `INPUT_PULLUP` dan `INPUT_PULLDOWN`. Perlu dicatat, pin input-only ESP32 (GPIO 34–39) tidak punya resistor pull internal sama sekali.

Debouncing

Kontak mekanik pada tombol bergetar saat bersentuhan atau terlepas, sehingga satu kali tekan bisa menghasilkan beberapa transisi HIGH–LOW dalam waktu sangat singkat (*bouncing*). Debouncing menunggu sinyal stabil selama waktu tertentu (misalnya 20–50 ms) sebelum sebuah perubahan dianggap sah. Nilai waktunya berupa kompromi: terlalu kecil masih lolos getaran, terlalu besar membuat tombol terasa lambat.

ADC

Selain digital output dan digital input, GPIO dapat berfungsi sebagai input analog lewat ADC (*Analog-to-Digital Converter*). ADC ESP32 memiliki resolusi default 12-bit (nilai 0–4095) dengan referensi sekitar 0–3.3 V, dibaca dengan `analogRead()`. ADC1 (misalnya GPIO 32–39) lebih disarankan daripada ADC2 karena ADC2 bermasalah saat WiFi aktif. Pada praktikum ini LDR dirangkai sebagai pembagi tegangan dengan resistor 10 kΩ, sehingga perubahan intensitas cahaya muncul sebagai perubahan tegangan di titik tengahnya.

2 Pendahuluan

2.1 Latar Belakang

Mikrokontroler hampir selalu ada di balik perangkat elektronik yang kita pakai sehari-hari, mulai dari remote AC, mesin cuci, sampai perangkat IoT yang terhubung ke internet. Karena itu, kemampuan memprogram mikrokontroler dan mengakses pin GPIO-nya menjadi pondasi sebelum kita bisa membuat sistem tertanam yang lebih kompleks.

Praktikum ini adalah langkah awal untuk membangun pondasi tersebut. Kita mulai dari menyiapkan lingkungan pengembangan (PlatformIO di VSCode), lalu mencoba dua board yang punya karakter

berbeda: STM32 Blackpill yang diprogram lewat ST-Link, dan ESP32 yang bisa diprogram langsung lewat USB. Pada ESP32 kita juga membandingkan dua framework, yaitu Arduino yang sederhana dan ESP-IDF yang lebih native, sehingga terlihat bahwa chip yang sama bisa didekati dengan cara yang berbeda.

Selain itu, praktikum ini membahas tiga cara kerja GPIO: sebagai output digital (menyalakan LED), sebagai input digital (membaca tombol), dan sebagai input analog (membaca sensor cahaya LDR lewat ADC). Di dunia nyata, tombol yang dibaca sembarangan akan menghasilkan data yang tidak bisa dipercaya, entah karena pin yang mengambang (*floating*) atau karena getaran kontak mekanik (*bouncing*). Memahami penyebab dan cara mengatasinya sejak awal membuat program yang kita tulis lebih andal, bukan sekadar jalan di meja percobaan.

2.2 Dasar Teori

PlatformIO dan VSCode

PlatformIO adalah ekosistem pengembangan *embedded* lintas platform yang mendukung banyak board dan framework (Arduino, STM32Cube/HAL, ESP-IDF, dan lainnya) dalam satu antarmuka. Board dan framework cukup diatur lewat berkas `platformio.ini`, sehingga satu chip seperti ESP32 bisa diprogram dengan Arduino atau ESP-IDF tanpa mengganti alat. Struktur project-nya terdiri dari `platformio.ini`, folder `src/` untuk kode utama, `include/` untuk header, dan `lib/` untuk library tambahan.

STM32 Blackpill

Blackpill berbasis STM32F401CC (ARM Cortex-M4 dengan FPU, clock hingga 84 MHz, Flash 256 KB, RAM 64 KB). LED onboard-nya berada di pin PC13 dan bersifat aktif LOW, artinya LED menyala saat pin diberi LOW. Board ini tidak punya programmer bawaan, jadi diperlukan ST-Link eksternal lewat protokol SWD (SWDIO, SWCLK, GND, dan 3.3V). Dengan framework Arduino (STM32duino), fungsi seperti `pinMode()`, `digitalWrite()`, dan `delay()` bisa dipakai seperti di Arduino biasa, meskipun di baliknya tetap berdiri di atas HAL dari STMicroelectronics.

ESP32 dan ESP-IDF

ESP32 adalah mikrokontroler 32-bit dual-core (Xtensa LX6) dengan WiFi dan Bluetooth terintegrasi, serta USB-to-Serial bawaan. ESP-IDF adalah framework resmi dari Espressif yang dibangun di atas FreeRTOS. Perbedaannya dengan framework Arduino dirangkum pada Tabel 2.

Tabel 2: Perbandingan framework Arduino dan ESP-IDF

Aspek	**Framework Arduino**	**Framework ESP-IDF**
Titik masuk program	`setup()` dan `loop()`	`app_main()`
Delay/timing	`delay(ms)`	`vTaskDelay(pdMS_TO_TICKS(ms))`
Tingkat abstraksi	Tinggi, mudah dipelajari	Rendah–menengah, lebih native dan kuat
Akses fitur RTOS	Tersembunyi/terbatas	Langsung (task, semaphore, queue)

Pull-up dan Pull-down

Pin GPIO yang tidak terhubung ke level tegangan apa pun akan *floating*, sehingga nilainya acak akibat *noise*. Untuk mengatasinya dipakai resistor pull-up (pin ditarik ke 3.3V, terbaca HIGH saat tombol tidak ditekan dan LOW saat ditekan ke GND) atau pull-down (pin ditarik ke GND, logikanya kebalikan). Resistor ini bisa dipasang eksternal (umumnya 10 kΩ) atau diaktifkan lewat firmware dengan `INPUT_PULLUP` dan `INPUT_PULLDOWN`. Perlu dicatat, pin input-only ESP32 (GPIO 34–39) tidak punya resistor pull internal sama sekali.

Debouncing

Kontak mekanik pada tombol bergetar saat bersentuhan atau terlepas, sehingga satu kali tekan bisa menghasilkan beberapa transisi HIGH–LOW dalam waktu sangat singkat (*bouncing*). Debouncing menunggu sinyal stabil selama waktu tertentu (misalnya 20–50 ms) sebelum sebuah perubahan dianggap sah. Nilai waktunya berupa kompromi: terlalu kecil masih lolos getaran, terlalu besar membuat tombol terasa lambat.

ADC

Selain digital output dan digital input, GPIO dapat berfungsi sebagai input analog lewat ADC (*Analog-to-Digital Converter*). ADC ESP32 memiliki resolusi default 12-bit (nilai 0–4095) dengan referensi sekitar 0–3.3 V, dibaca dengan `analogRead()`. ADC1 (misalnya GPIO 32–39) lebih disarankan daripada ADC2 karena ADC2 bermasalah saat WiFi aktif. Pada praktikum ini LDR dirangkai sebagai pembagi tegangan dengan resistor 10 kΩ, sehingga perubahan intensitas cahaya muncul sebagai perubahan tegangan di titik tengahnya.

3 Tugas Pendahuluan

Berikut jawaban tugas pendahuluan beserta penjelasannya.

1. **Perbedaan mendasar framework Arduino dan ESP-IDF, beserta kelebihan dan kekurangannya.**

   Arduino adalah framework dengan tingkat abstraksi tinggi. Programnya cukup berisi `setup()` yang berjalan sekali dan `loop()` yang berulang, dengan fungsi sederhana seperti `pinMode()`, `digitalWrite()`, dan `delay()`. ESP-IDF adalah framework resmi dari Espressif yang dibangun di atas FreeRTOS. Program dimulai dari `app_main()` yang berjalan sebagai *task*, dan penundaan memakai `vTaskDelay(pdMS_TO_TICKS(ms))`. Perbedaan ini membuat ESP-IDF memberi akses langsung ke fitur RTOS (task, semaphore, queue) dan konfigurasi *low-level*, sementara pada Arduino hal tersebut tersembunyi atau terbatas.

   Kelebihan Arduino adalah kurva belajarnya landai, kodenya singkat, dan library-nya banyak, sehingga cocok untuk pemula dan prototipe cepat. Kekurangannya, kontrol terhadap detail hardware dan penjadwalan task terbatas, dan abstraksinya menyembunyikan cara kerja di bawahnya. Kelebihan ESP-IDF adalah kontrol penuh, performa dan manajemen sumber daya yang lebih baik, serta cocok untuk produk IoT tingkat lanjut. Kekurangannya, kode lebih panjang, konsep seperti RTOS harus dipahami dulu, dan butuh waktu belajar lebih lama.

2. **Mengapa STM32 Blackpill memerlukan ST-Link, sedangkan ESP32 bisa diprogram langsung lewat USB?**

   Komputer berkomunikasi lewat protokol USB, sedangkan jalur pemrograman ESP32 berupa UART (serial). Board ESP32 DevKit sudah dilengkapi chip *USB-to-Serial* (CP2102 atau CH340) yang menerjemahkan sinyal USB dari komputer menjadi sinyal UART yang dimengerti ESP32. ESP32 sendiri punya *bootloader* bawaan yang menerima program baru lewat UART, sehingga kabel USB saja sudah cukup untuk melakukan *flashing*.

   Board Blackpill tidak membawa chip penerjemah seperti itu. Jalur yang dipakai adalah protokol SWD (SWDIO, SWCLK, GND, dan 3.3V), dan komputer tidak bisa berbicara SWD secara langsung lewat USB. Karena itu dibutuhkan ST-Link sebagai jembatan: sisi USB-nya dihubungkan ke komputer, sisi SWD-nya dihubungkan ke Blackpill. Singkatnya, perbedaannya terletak pada ada atau tidaknya penerjemah USB di dalam board itu sendiri.

3. **Kondisi logika pada konfigurasi pull-up dan pull-down.**

   Pada **pull-up**, pin GPIO ditarik ke 3.3V lewat resistor, dan tombol dipasang antara pin dan GND. Saat tombol *tidak ditekan*, pin terbaca **HIGH** karena tertarik ke 3.3V. Saat tombol *ditekan*, pin terhubung langsung ke GND sehingga terbaca **LOW**.

   Pada **pull-down**, pin GPIO ditarik ke GND lewat resistor, dan tombol dipasang antara pin dan 3.3V. Saat tombol *tidak ditekan*, pin terbaca **LOW**. Saat tombol *ditekan*, pin terhubung ke 3.3V sehingga terbaca **HIGH**. Jadi logikanya berkebalikan dengan pull-up, seperti dirangkum pada Tabel 3.

   Tabel 3: Kondisi logika pull-up dan pull-down

Konfigurasi	Tombol tidak ditekan	Tombol ditekan
Pull-up	HIGH	LOW
Pull-down	LOW	HIGH

4. **Fenomena bouncing pada tombol mekanik dan cara mengatasinya.**

   Bouncing adalah kondisi ketika satu kali penekanan atau pelepasan tombol menghasilkan beberapa transisi HIGH–LOW dalam waktu sangat singkat. Penyebabnya adalah sifat fisik kontak logam di dalam tombol. Saat dua kontak bertemu atau berpisah, kontak itu tidak langsung diam, tetapi memantul dan bergetar sesaat sehingga sambungannya terputus-sambung berulang kali sebelum stabil. Mikrokontroler membaca pin dengan sangat cepat, jadi getaran ini terbaca sebagai banyak penekanan, padahal tombol hanya ditekan sekali.

   Beberapa pendekatan untuk mengatasinya:

   - **Software, berbasis waktu.** Perubahan sinyal baru dianggap sah jika sudah stabil selama waktu tertentu (misalnya 20–50 ms), dengan mencatat waktu perubahan menggunakan `millis()`. Cara ini dipakai pada Percobaan 4. Metode paling sederhana adalah `delay()` singkat setelah mendeteksi tekanan, tetapi ini memblokir program.
   - **Hardware, filter RC.** Kapasitor dipasang paralel dengan tombol bersama resistor sehingga lonjakan cepat akibat getaran diredam. Sering dikombinasikan dengan *Schmitt trigger* agar transisinya tegas.

- **Hardware, rangkaian SR latch.** Sebuah *latch* menjaga keluaran tetap bersih meskipun kontak bergetar.

5. **Cara kerja ADC dan pengaruh resolusi bit terhadap presisi.**

   ADC mengubah tegangan analog yang kontinu menjadi angka digital lewat tiga tahap. Pertama, *sampling*: tegangan pada pin diambil (dicuplik) pada saat tertentu. Kedua, *kuantisasi*: nilai tegangan itu dipetakan ke salah satu dari sejumlah tingkat diskret yang tersedia, dengan pembulatan ke tingkat terdekat. Ketiga, *pengkodean*: tingkat tersebut dinyatakan sebagai bilangan biner. Hasilnya berbanding lurus dengan tegangan masukan terhadap tegangan referensi:

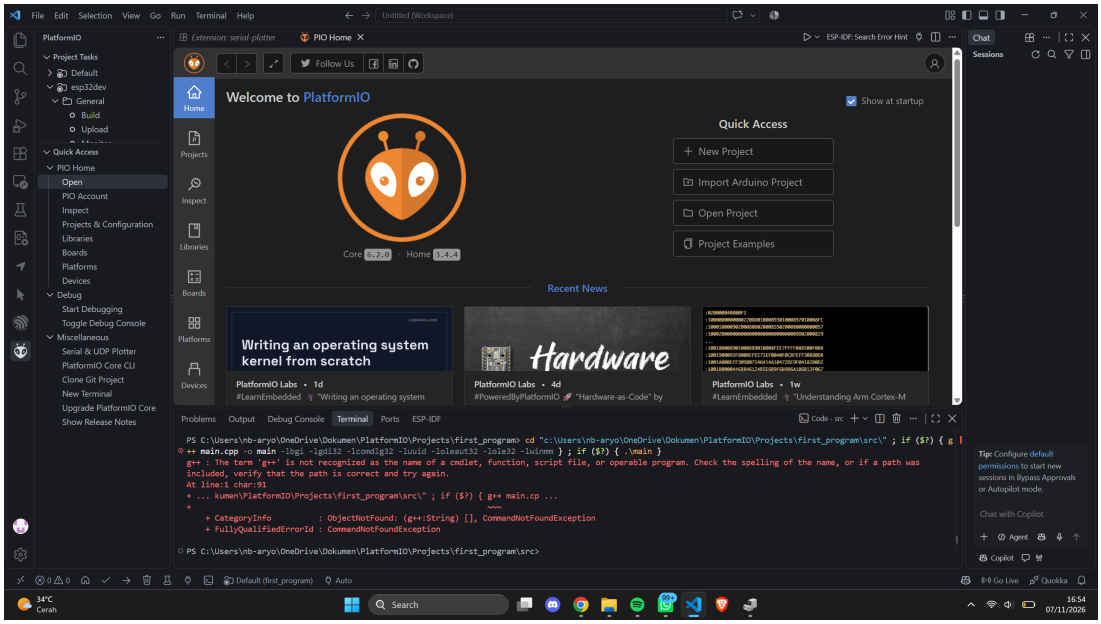
$$\text{nilai ADC} \approx \frac{V_{in}}{V_{ref}} \times (2^n - 1) \tag{1}$$

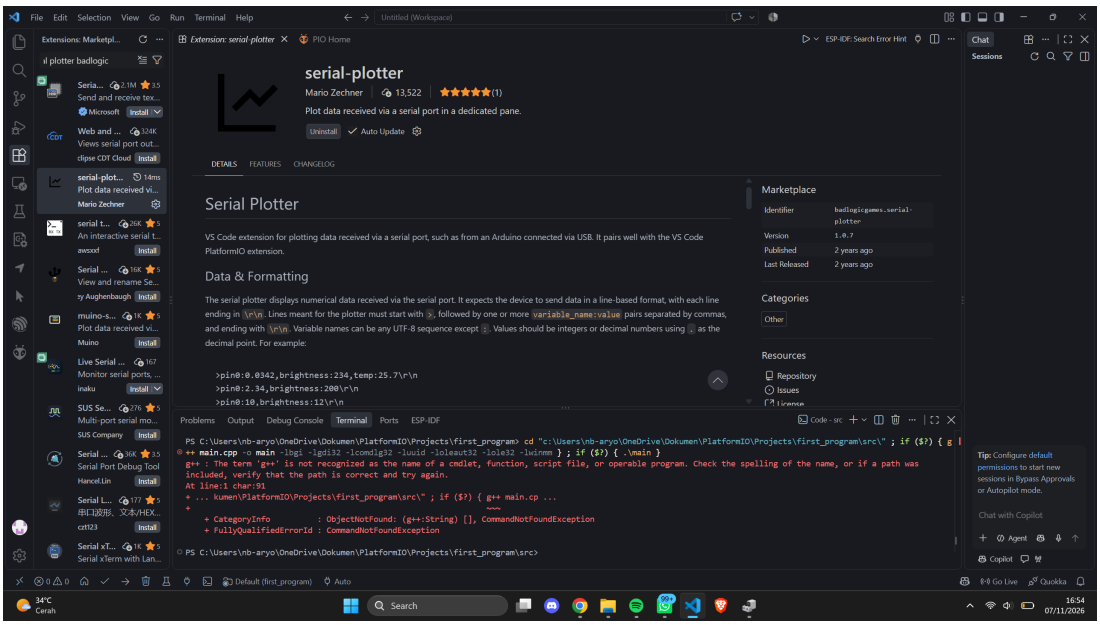
   dengan n adalah resolusi dalam bit.

   Resolusi menentukan berapa banyak tingkat yang bisa dibedakan, yaitu 2^n tingkat, dan ukuran satu langkahnya (*LSB*) kira-kira $V_{ref}/2^n$. Makin besar n, makin halus langkahnya dan makin presisi pembacaannya. Sebagai contoh dengan $V_{ref} = 3.3$ V, ADC 10-bit memiliki 1024 tingkat dengan langkah sekitar 3.2 mV, sedangkan ADC 12-bit milik ESP32 memiliki 4096 tingkat (nilai 0–4095) dengan langkah sekitar 0.8 mV. Jadi perubahan tegangan yang kecil masih bisa terdeteksi pada resolusi yang lebih tinggi.

6. **Set-up tools: VSCode, PlatformIO (ESP-IDF dan STM32), dan Serial Plotter.**

   Seluruh tools sudah terpasang di laptop. Langkah yang dilakukan: (1) memasang Visual Studio Code, (2) memasang ekstensi PlatformIO IDE lalu membuat project dengan platform `espressif32` (framework ESP-IDF) dan `ststm32` (framework Arduino/STM32duino), dan (3) memasang ekstensi Serial Plotter untuk menampilkan data serial dalam bentuk grafik. Driver USB (ST-Link untuk STM32 serta CP2102/CH340 untuk ESP32) juga sudah dipasang. Bukti set-up ditunjukkan pada Gambar 1.





Gambar 1: Tangkapan layar set-up VSCode, PlatformIO, dan Serial Plotter